

STL 2

Escaneie para acessar o slide da Aula 4



luisenilton.github.io/Curso-Maratona/aula4

Sumário da Aula

- | | |
|---------------------------------------|---------------------------------------|
| 1. <u>STL 2</u> | 12. <u>Tabela Hash</u> |
| 2. <u>Sumário da Aula</u> | 13. <u>Requisitos para uma função</u> |
| 3. <u>Recapitulando Busca Binária</u> | <u>Hash</u> |
| 4. <u>LowerBound e UpperBound</u> | 14. <u>Colisões</u> |
| 5. <u>LowerBound e UpperBound</u> | 15. <u>Tabela Hash em C++</u> |
| <u>em C++</u> | 16. <u>Unordered Set</u> |
| 6. <u>Aplicações</u> | 17. <u>Fila de Prioridade</u> |
| 7. <u>Arvore Binária</u> | 18. <u>Referências</u> |
| 8. <u>Arvore de busca binária</u> | 19. <u>Contatos</u> |
| 9. <u>Map</u> | 20. <u>Obrigado por acompanhar a</u> |
| 10. <u>Set</u> | <u>aula</u> |
| 11. <u>Multiset</u> | |

Recapitulando Busca Binária

Recapitulando Busca Binária

Dado um array ordenado conseguimos encontrar a posição de um elemento X em $O(\log n)$, basta ir dividindo o array no meio

Recapitulando Busca Binária

Dado um array ordenado conseguimos encontrar a posição de um elemento X em $O(\log n)$, basta ir dividindo o array no meio

```
int esquerda = 0, direita = a.size() - 1;
while(esquerda ≤ direita){
    int meio = (esquerda + direita)/2;
    if(a[meio] == x) return true;
    else if(a[meio] < x){
        esquerda = meio + 1;
    }else{
        direita = meio - 1;
    }
}
return false;
```

LowerBound e UpperBound

LowerBound e UpperBound

Como encontrar o primeiro elemento $\geq x$ num array ordenado, isso é o `lower_bound(x)` num array.
`upper_bound(x)` é o primeiro elemento $> x$.

LowerBound e UpperBound

Como encontrar o primeiro elemento $\geq x$ num array ordenado, isso é o `lower_bound(x)` num array. `upper_bound(x)` é o primeiro elemento $> x$.

A primeira intuição é percorrer o array até achar um cara $\geq x$, porém a complexidade disso é $O(N)$

LowerBound e UpperBound

Como encontrar o primeiro elemento $\geq x$ num array ordenado, isso é o `lower_bound(x)` num array. `upper_bound(x)` é o primeiro elemento $> x$.

A primeira intuição é percorrer o array até achar um cara $\geq x$, porém a complexidade disso é $O(N)$

```
int idx = 0;
for(; idx < n; idx++)
    if(a[idx] ≥ x) break;
```

LowerBound e UpperBound

Como encontrar o primeiro elemento $\geq$ a X num array ordenado, isso é o `lower_bound(x)` num array. `upper_bound(x)` é o primeiro elemento $> x$.

A primeira intuição é percorrer o array até achar um cara $\geq$ a X , porém a complexidade disso é $O(N)$

```
int idx = 0;
for(; idx < n; idx++)
    if(a[idx] ≥ X) break;
```

Podemos resolver de forma mais rápida utilizando busca binária, a ideia é que podemos cortar uma metade do array quando chutamos um numero, se o elementor é menor do que X então o `lower_bound` está para a direita, se for maior ou igual, marcamos essa posição como resposta e vamos para a esquerda. $O(\log N)$

LowerBound e UpperBound

Como encontrar o primeiro elemento $\geq$ a X num array ordenado, isso é o `lower_bound(x)` num array. `upper_bound(x)` é o primeiro elemento $> x$.

A primeira intuição é percorrer o array até achar um cara $\geq$ a X , porém a complexidade disso é $O(N)$

```
int idx = 0;
for(;idx < n;idx++)
    if(a[idx] ≥ X) break;
```

Podemos resolver de forma mais rápida utilizando busca binária, a ideia é que podemos cortar uma metade do array quando chutamos um numero, se o elementor é menor do que X então o `lower_bound` está para a direita, se for maior ou igual, marcamos essa posição como resposta e vamos para a esquerda. $O(\log N)$

```
int esquerda = 0, direita = a.size() - 1;
int lb = -1;
while(esquerda ≤ direita){
    int meio = (esquerda + direita)/2;
    if(a[meio] ≥ x){
        lb = meio;
        r = m - 1;
    }else{
        l = m + 1;
    }
}
```

LowerBound e UpperBound em C++

LowerBound e UpperBound em C++

No STL já existem funções para calcular o `lower_bound` e `upper_bound`, elas retornam um iterador para a posição. A complexidade é $O(\log n)$.

LowerBound e UpperBound em C++

No STL já existem funções para calcular o `lower_bound` e `upper_bound`, elas retornam um iterador para a posição. A complexidade é $O(\log n)$.

```
vector<int> a = {1,2,3,4,5};  
int x = 2;  
auto it_lb = lower_bound(a.begin(),a.end(),x);  
auto it_ub = upper_bound(a.begin(),a.end(),x);  
int indice = it_lb - a.begin(); // pra descobrir o indice subtrae do .begin()
```

LowerBound e UpperBound em C++

No STL já existem funções para calcular o `lower_bound` e `upper_bound`, elas retornam um iterador para a posição. A complexidade é $O(\log n)$.

```
vector<int> a = {1,2,3,4,5};  
int x = 2;  
auto it_lb = lower_bound(a.begin(),a.end(),x);  
auto it_ub = upper_bound(a.begin(),a.end(),x);  
int indice = it_lb - a.begin(); // pra descobrir o indice subtrae do .begin()
```

Duvida

LowerBound e UpperBound em C++

No STL já existem funções para calcular o `lower_bound` e `upper_bound`, elas retornam um iterador para a posição. A complexidade é $O(\log n)$.

```
vector<int> a = {1,2,3,4,5};  
int x = 2;  
auto it_lb = lower_bound(a.begin(),a.end(),x);  
auto it_ub = upper_bound(a.begin(),a.end(),x);  
int indice = it_lb - a.begin(); // pra descobrir o indice subtrae do .begin()
```

Duvida

Como encontrar o primeiro elemento $\leq$ e $<$ a X ?

LowerBound e UpperBound em C++

No STL já existem funções para calcular o `lower_bound` e `upper_bound`, elas retornam um iterador para a posição. A complexidade é $O(\log n)$.

```
vector<int> a = {1,2,3,4,5};  
int x = 2;  
auto it_lb = lower_bound(a.begin(),a.end(),x);  
auto it_ub = upper_bound(a.begin(),a.end(),x);  
int indice = it_lb - a.begin(); // pra descobrir o indice subtrae do .begin()
```

Duvida

Como encontrar o primeiro elemento $\leq$ e $<$ a X ?

Tente utilizar o `lower_bound` e `upper_bound` que já estão implementados.

LowerBound e UpperBound em C++

No STL já existem funções para calcular o `lower_bound` e `upper_bound`, elas retornam um iterador para a posição. A complexidade é $O(\log n)$.

```
vector<int> a = {1,2,3,4,5};  
int x = 2;  
auto it_lb = lower_bound(a.begin(),a.end(),x);  
auto it_ub = upper_bound(a.begin(),a.end(),x);  
int indice = it_lb - a.begin(); // pra descobrir o indice subtrae do .begin()
```

Duvida

Como encontrar o primeiro elemento $\leq$ e $<$ a X ?

Tente utilizar o `lower_bound` e `upper_bound` que já estão implementados.

O primeiro elemento $<$ do que X está uma posição atrás do primeiro elemento $\geq$ a X.

Aplicações

Aplicações

- Contar quantas aparições X tem no array.

Aplicações

- Contar quantas aparições X tem no array.
- Dado dois valores ``a`` e ``b`` ($a \leq b$), descubra quantos elementos tem valor entre ``a`` e ``b``

Aplicações

- Contar quantas aparições X tem no array.
- Dado dois valores ``a`` e ``b`` ($a \leq b$), descubra quantos elementos tem valor entre ``a`` e ``b``
- Questão

Árvore Binária

Árvore Binária

O que é árvore?

Arvore Binária

O que é arvore?

Estrutura de dados não linear que consiste em nós conectados por arestas, dizemos que ela tem uma raiz e que cada nó tem seus filhos.

Árvore Binária

O que é árvore?

Estrutura de dados não linear que consiste em nós conectados por arestas, dizemos que ela tem uma raiz e que cada nó tem seus filhos.

É utilizada para resolver problemas que estruturas de dados linear demoram, são a base para containers associativos.

Árvore Binária

O que é árvore?

Estrutura de dados não linear que consiste em nós conectados por arestas, dizemos que ela tem uma raiz e que cada nó tem seus filhos.

É utilizada para resolver problemas que estruturas de dados linear demoram, são a base para containers associativos.

Binária?

Árvore Binária

O que é árvore?

Estrutura de dados não linear que consiste em nós conectados por arestas, dizemos que ela tem uma raiz e que cada nó tem seus filhos.

É utilizada para resolver problemas que estruturas de dados linear demoram, são a base para containers associativos.

Binária?

Uma árvore binária é uma estrutura de árvore em que cada nó pode ter no máximo dois filhos.

Arvore Binária

O que é arvore?

Estrutura de dados não linear que consiste em nós conectados por arestas, dizemos que ela tem uma raiz e que cada nó tem seus filhos.

É utilizada para resolver problemas que estruturas de dados linear demoram, são a base para containers associativos.

Binária?

Uma arvore binária é uma estrutura de arvore em que cada nó pode ter no máximo dois filhos.

Seu objetivo é permitir a inserção e pesquisa de elementos em $O(\log n)$, mantendo a ordenação.

Arvore Binária

O que é arvore?

Estrutura de dados não linear que consiste em nós conectados por arestas, dizemos que ela tem uma raiz e que cada nó tem seus filhos.

É utilizada para resolver problemas que estruturas de dados linear demoram, são a base para containers associativos.

Binária?

Uma arvore binária é uma estrutura de arvore em que cada nó pode ter no máximo dois filhos.

Seu objetivo é permitir a inserção e pesquisa de elementos em $O(\log n)$, mantendo a ordenação.

Um conceito importante é o **balanceamento**, uma arvore é dita balanceada quando se pra cada um de seus nós o tamanho da subarvore da esquerda e da direita é igual (ou difere apenas por 1).

Arvore Binária

O que é arvore?

Estrutura de dados não linear que consiste em nós conectados por arestas, dizemos que ela tem uma raiz e que cada nó tem seus filhos.

É utilizada para resolver problemas que estruturas de dados linear demoram, são a base para containers associativos.

Binária?

Uma arvore binária é uma estrutura de arvore em que cada nó pode ter no máximo dois filhos.

Seu objetivo é permitir a inserção e pesquisa de elementos em $O(\log n)$, mantendo a ordenação.

Um conceito importante é o **balanceamento**, uma arvore é dita balanceada quando se pra cada um de seus nós o tamanho da subarvore da esquerda e da direita é igual (ou difere apenas por 1).

Visualização

Árvore de busca binária

Arvore de busca binária

Uma das principais aplicações de arvores binárias é criar uma estrutura que permita realizar busca binária num conjuntos de elementos, além de permitir a inserção e remoção em tempo real, tudo isso em $\text{Log}(n)$.

Árvore de busca binária

Uma das principais aplicações de árvores binárias é criar uma estrutura que permita realizar busca binária num conjunto de elementos, além de permitir a inserção e remoção em tempo real, tudo isso em $\text{Log}(n)$.

Criaremos uma árvore em a esquerda de nó só tenha nós com valores menores que ele e para direita apenas nós com valores maiores do que ele.

Arvore de busca binária

Uma das principais aplicações de arvores binárias é criar uma estrutura que permita realizar busca binária num conjuntos de elementos, além de permitir a inserção e remoção em tempo real, tudo isso em $\text{Log}(n)$.

Criaremos uma arvore em a esquerda de nó só tenha nós com valores menores que ele e para direita apenas nós com valores maiores do que ele.

A ideia consiste em colocar o elemento central do array como raiz da arvore, agora cada chute que você daria para a esquerda ou para direita serão os filhos desse nó, isso repete para cada nó.

Arvore de busca binária

Uma das principais aplicações de arvores binárias é criar uma estrutura que permita realizar busca binária num conjuntos de elementos, além de permitir a inserção e remoção em tempo real, tudo isso em $\text{Log}(n)$.

Criaremos uma arvore em a esquerda de nó só tenha nós com valores menores que ele e para direita apenas nós com valores maiores do que ele.

A ideia consiste em colocar o elemento central do array como raiz da arvore, agora cada chute que você daria para a esquerda ou para direita serão os filhos desse nó, isso repete para cada nó.

Exemplo: 1,2,3,4,5,6,7.

Arvore de busca binária

Uma das principais aplicações de arvores binárias é criar uma estrutura que permita realizar busca binária num conjunto de elementos, além de permitir a inserção e remoção em tempo real, tudo isso em $\text{Log}(n)$.

Criaremos uma arvore em a esquerda de nó só tenha nós com valores menores que ele e para direita apenas nós com valores maiores do que ele.

A ideia consiste em colocar o elemento central do array como raiz da arvore, agora cada chute que você daria para a esquerda ou para direita serão os filhos desse nó, isso repete para cada nó.

Exemplo: 1,2,3,4,5,6,7.

Para o tempo de execução ser $O(\log N)$ a arvore tem que está balanceada. Porque?

Map

Maps são containers associativos do SLT que guarda pares de valores no formato (chave, valor), a chave é unica, mas o valor não precisa ser. Muito útil em que você precisa guarda informações para cada pessoa, id e etc.

```
map<string,int> mapa; // mapa<TIPO_CHAVE, TIPO_VALOR>
string nome = "abc";
mapa[nome] = 2;
cout << mapa[nome] << endl; // pra acessar o valor de um chave em O(logn)
cout << st.count(3) << endl; // podemos saber se um chave está no mapa em O(logn)
mapa.erase(nome) // podemos apagar um valor dado uma chave em O(logN).
cout << st.size() << endl; // retorna o tamanho do mapa, quantas chaves
```

Questão

Set

Sets são containers associativos do STL que guarda elementos únicos de um mesmo tipo de maneira ordenada. São implementados usando árvore binária por baixo dos panos.

```
set<int> st; // set<TIPO>

// podemos inserir elementos em  $O(\log N)$ 
st.insert(3); // {3}
st.insert(2); // {2,3}
st.insert(3); // {2,3}

cout << st.count(3) << endl; // podemos verificar se um elemento está no conjunto em  $O(\log N)$ .
st.erase(3) // podemos apagar um valor do conjunto em  $O(\log N)$ 
cout << st.size() << endl; // retorna o tamanho do conjunto
```

Questão

Multiset

Multisets são containers semelhante ao set, porém permitem elementos duplicados.

```
multiset<int> mst = {2,3,3,4,5}; // multiset<TIP0>

cout << mst.count(3) << endl; // podemos verificar se um elementos está no conjunto em O(logN).
mst.erase(3) // !!!Cuidado com o erase, isso apaga todas as ocorrencias do 3
mst.erase(mst.find(3)); // jeito certo de apagar um elemento

auto it = mst.lower_bound(2); // tambem podemos achar o lower_bound de um valor, retorna uma referencia
cout << *it << endl;
it = mst.upper_bound(3);
```

Questão

Tabela Hash

Tabela Hash

Como vimos anteriormente, a complexidade para consultar um índice num array é $O(1)$, porém um índice num array é sempre um número inteiro.

Tabela Hash

Como vimos anteriormente, a complexidade para consultar um índice num array é $O(1)$, porém um índice num array é sempre um número inteiro.

Isso nos traz uma motivação, e se tivéssemos um array em que os índices são strings ou double ou qualquer coisa. Como faríamos isso?

Tabela Hash

Como vimos anteriormente, a complexidade para consultar um índice num array é $O(1)$, porém um índice num array é sempre um número inteiro.

Isso nos traz uma motivação, e se tivéssemos um array em que os índices são strings ou double ou qualquer coisa. Como faríamos isso?

Especificação

Tabela Hash

Como vimos anteriormente, a complexidade para consultar um índice num array é $O(1)$, porém um índice num array é sempre um número inteiro.

Isso nos traz uma motivação, e se tivéssemos um array em que os índices são strings ou double ou qualquer coisa. Como faríamos isso?

Especificação

Queremos uma estrutura que funcione para qualquer tipo e tenha as seguintes operações

Tabela Hash

Como vimos anteriormente, a complexidade para consultar um índice num array é $O(1)$, porém um índice num array é sempre um número inteiro.

Isso nos traz uma motivação, e se tivéssemos um array em que os índices são strings ou double ou qualquer coisa. Como faríamos isso?

Especificação

Queremos uma estrutura que funcione para qualquer tipo e tenha as seguintes operações

- $\text{Get}(K)$, recuperar o valor salvo com esta chave em $O(1)$.
- $\text{Insert}(K,V)$, criar uma posição no mapa que atribui o valor V a chave K em $O(1)$.

Tabela Hash

Como vimos anteriormente, a complexidade para consultar um índice num array é $O(1)$, porém um índice num array é sempre um número inteiro.

Isso nos traz uma motivação, e se tivéssemos um array em que os índices são strings ou double ou qualquer coisa. Como faríamos isso?

Especificação

Queremos uma estrutura que funcione para qualquer tipo e tenha as seguintes operações

- $\text{Get}(K)$, recuperar o valor salvo com esta chave em $O(1)$.
- $\text{Insert}(K,V)$, criar uma posição no mapa que atribui o valor V a chave K em $O(1)$.

Para resolver esse problema utilizamos tabelas Hash, eles funcionam mapeando qualquer tipo através de uma função hash para posições num array.

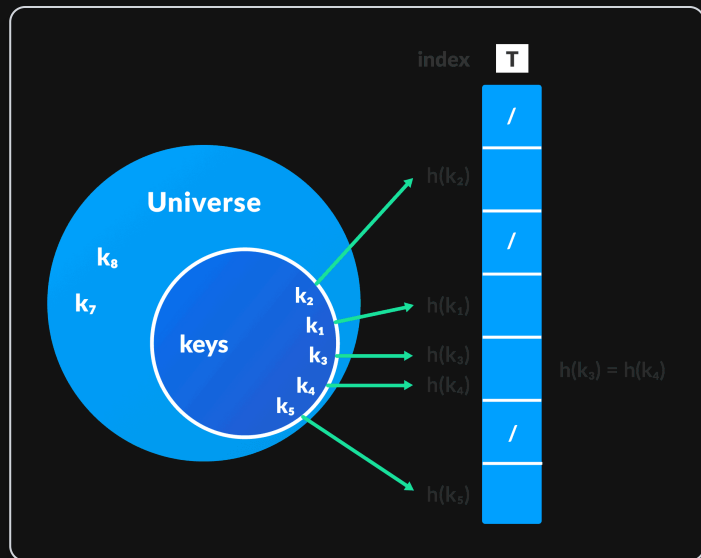
Requisitos para uma função Hash

Requisitos para uma função Hash

- Ela deve ser consistente, para um mesmo valor ela sempre retorna o mesmo número.
- Deve mapear diferentes valores para posições diferentes.
- Matematicamente, uma função hash perfeita é bijetora.

Requisitos para uma função Hash

- Ela deve ser consistente, para um mesmo valor ela sempre retorna o mesmo número.
- Deve mapear diferentes valores para posições diferentes.
- Matematicamente, uma função hash perfeita é bijetora.



Colisões

Na prática é inevitável que 2 valores retornem a mesma posição em uma função hash, o que chamamos de colisão.

Para resolver isso criamos uma lista ligada em cada posição, e fazemos uma busca sequencial ao recuperar um elemento.

Visualização

Tabela Hash em C++

Em C++, a tabela hash é implementada no `unordered_map`, ela tem os mesmos métodos do `map`, exceto por `lower_bound` e `upper_bound`, mas roda em $O(1)$.

Na teoria, o pior caso da tabela é $O(N)$, porém na prática ela é bem mais rápida e assumimos como $O(1)$.

```
unordered_map<string,int> mapa; // unordered_map<TIPO_CHAVE, TIPO_VALOR>
string nome = "abc";
mapa[nome] = 2;
cout << mapa[nome] << endl; // pra acessar o valor de um chave em  $O(1)$ 
cout << st.count(3) << endl; // podemos saber se um chave está no mapa em  $O(1)$ 
mapa.erase(nome) // podemos apagar um valor dado uma chave em  $O(1)$ .
cout << st.size() << endl; // retorna o tamanho do mapa, quantas chaves
```

[Link Problema](#)

Unordered Set

Existe uma versão de set em C++ que usa tabela hash por baixo dos panos, então permite inserção e consulta em $O(1)$.

Fica a critério do desenvolvedor escolher qual dos dois usar, a vantagem do set normal é manter ordenado, e a do unordered é ser $O(1)$.

```
unordered_set<int> st; // unordered_set<TIPO>

// podemos inserir elementos em  $O(\log N)$ 
st.insert(3); // {3}
st.insert(2); // {2,3}
st.insert(3); // {2,3}

cout << st.count(3) << endl; // podemos verificar se um elementos está no conjunto em  $O(\log N)$ .
st.erase(3) // podemos apagar um valor do conjunto em  $O(\log N)$ 
cout << st.size() << endl; // retorna o tamanho do conjunto
```

Fila de Prioridade

A fila de prioridade é uma estrutura que permite inserir elementos numa fila, porém o elemento da frente é sempre o maior que entrou.

Ela é muito utilizada em situações em que elementos são inseridos em tempo real e você quer saber qual o maior elemento até agora.

Um multiset consegue fazer tudo que uma fila de prioridade faz, porém ela executa mais rápido por usar um Heap ao invés de árvore binária.

```
priority_queue<int> pq;

pq.push(1);
cout << pq.top() << endl; // 1
pq.push(2);
cout << pq.top() << endl; // 2
pq.push(3);
cout << pq.top() << endl; // 3

// se você quiser uma fila que o menor fique sempre no topo
priority_queue<int,vector<int>,greater<int>> pq;
```

Falaremos mais sobre essa estrutura na aula sobre algoritmos gulosos.

Referências

- Documentação C++
- Programiz DSA
- Livro CPH
- VisuAlgo
- Entendendo Algoritmos

Contatos

Entre no discord para dúvidas



Entre no grupo de questões



Obrigado por acompanhar a aula

Por favor preencha o formulário de feedback com sugestões e críticas para a próxima aula 🙏.

